UNIT-2
LINEAR DATA STRUCTURE

PRESENTED BY

S.SWAIN

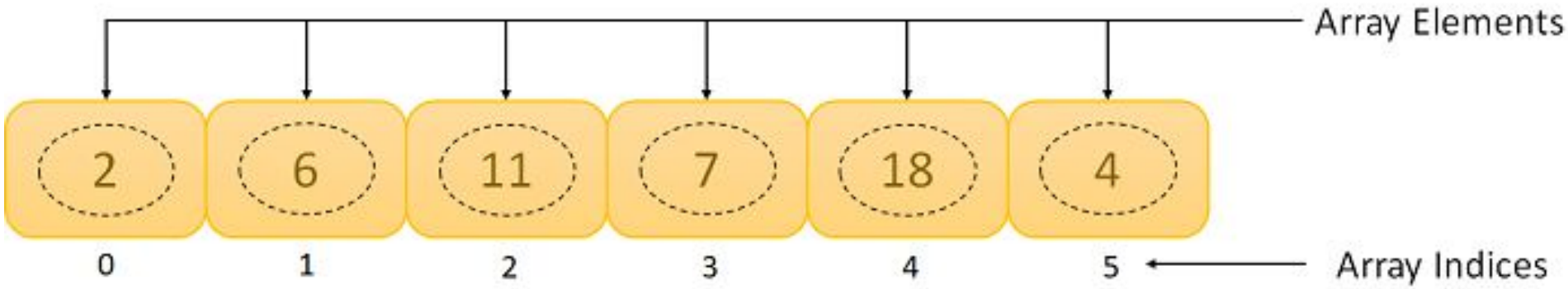
Array

An Array is a list of elements where each element has a unique place in the list.

The data elements of the array share the same variable name; however, each carries a different index number called a **subscript**.

We can access any data element from the list with the help of its location in the list.

Thus, the key feature of the arrays to understand is that **the data is stored in contiguous memory locations, making it possible for the users to traverse through the data elements of the array using their respective indexes.**



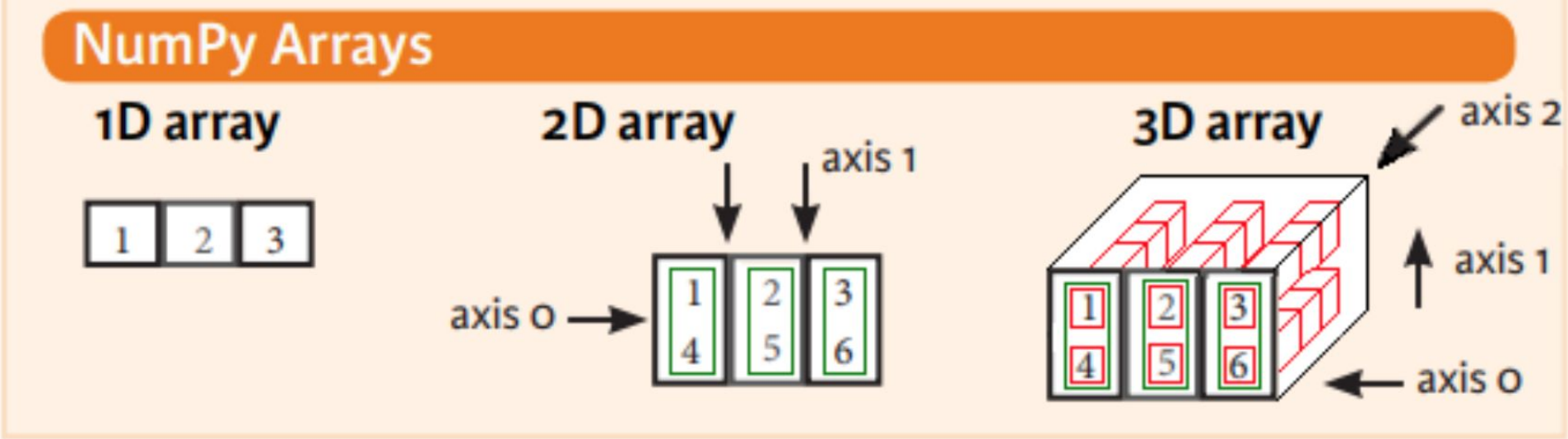
Arrays can be classified into different types:

One-Dimensional Array: An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.

Two-Dimensional Array: An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.

Multidimensional Array: We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices are per the need .

EXAMPLE


NumPy Arrays
1D array
1
2
3
2D array
axis 1
axis 0
1
4
2
5
3
6
3D array
axis 2
axis 1
axis 0
1
4
2
5
3
6

Some Applications of Array:

We can store a list of data elements belonging to the same data type.

Array acts as an auxiliary storage for other data structures.

The array also helps store data elements of a binary tree of the fixed count.

Array also acts as a storage of matrices.

Selection Short

Selection sort is a simple sorting algorithm.

This sorting algorithm, like insertion sort, is an in-place comparison-based algorithm in which the list is **divided into two parts, the sorted part at the left end and the unsorted part at the right end.**

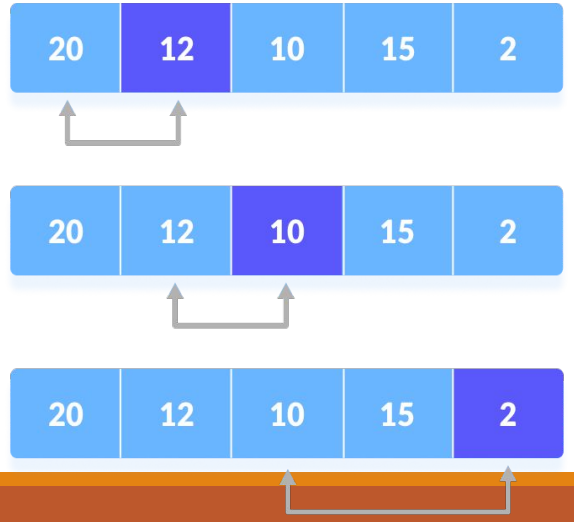
Initially, **the sorted part is empty and the unsorted part is the entire list.**

This algorithm is not suitable for large data sets as its average and worst case complexities are of **$O(n^2)$**, where **n** is the number of items.

Working of Selection Sort

Set the first element as minimum

Compare minimum with the second element. If the second element is smaller than minimum, assign the second element as minimum

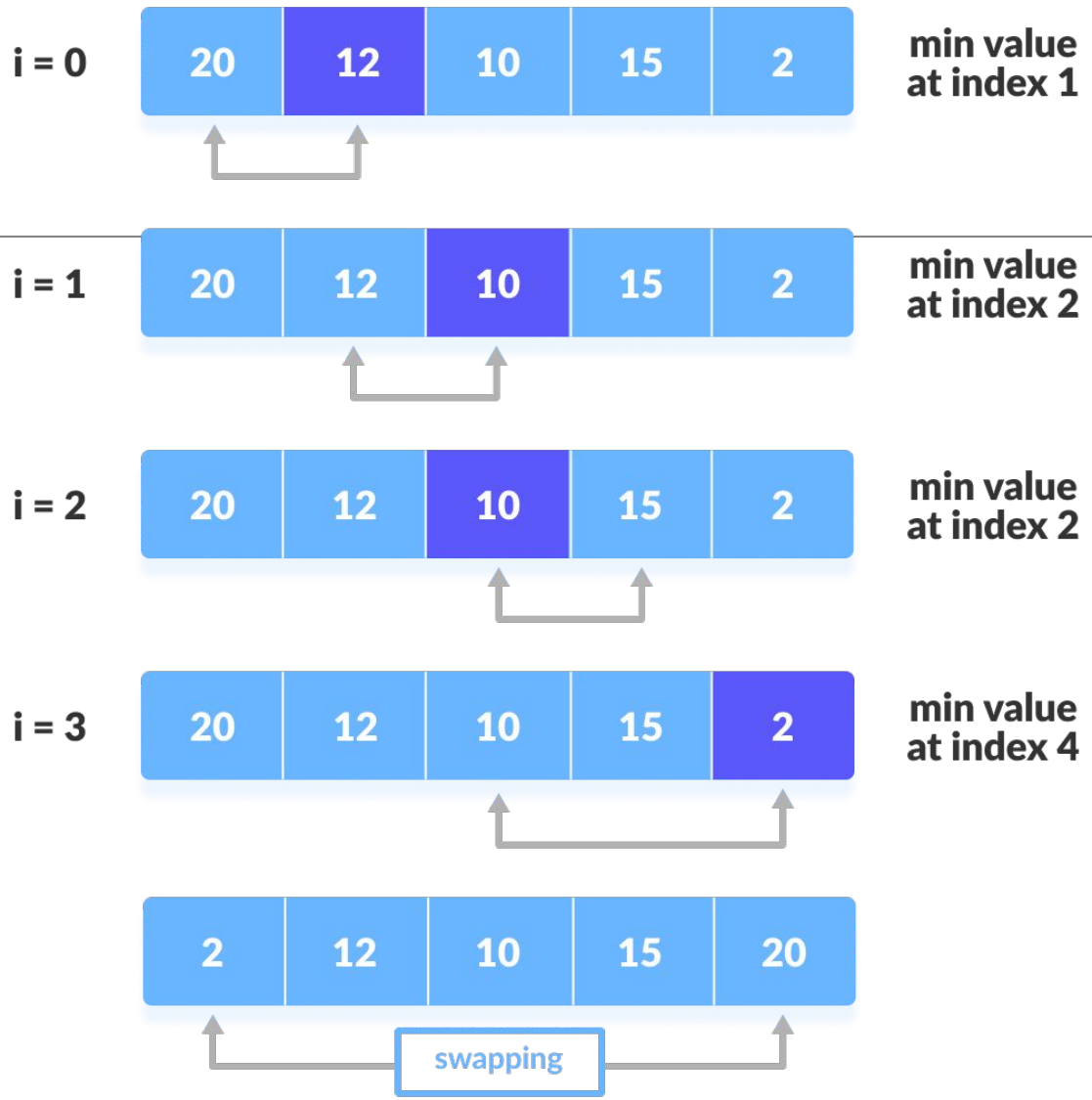


After each iteration, minimum is placed in the front o

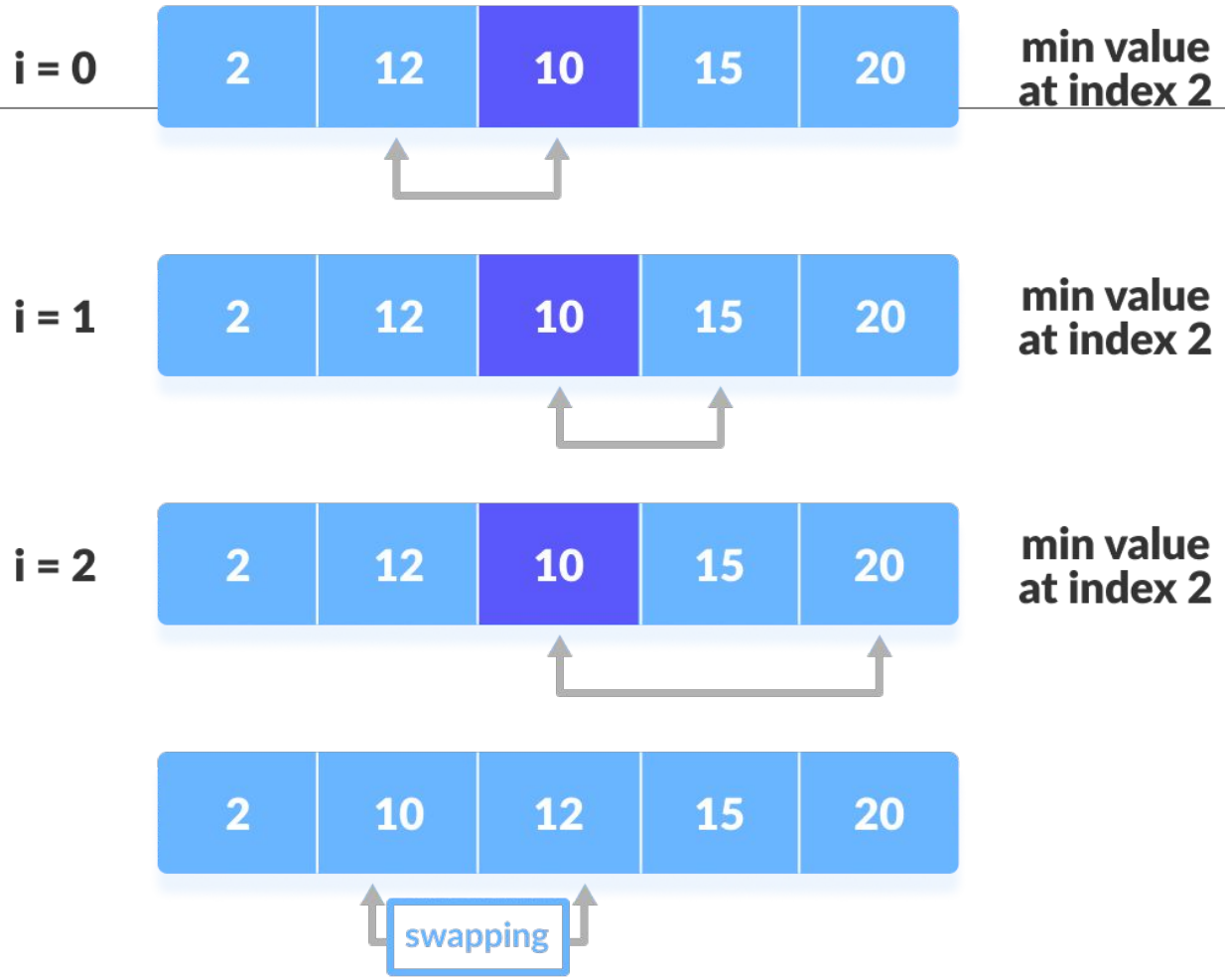
If the unsorted list first with minimum then swap.

For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions

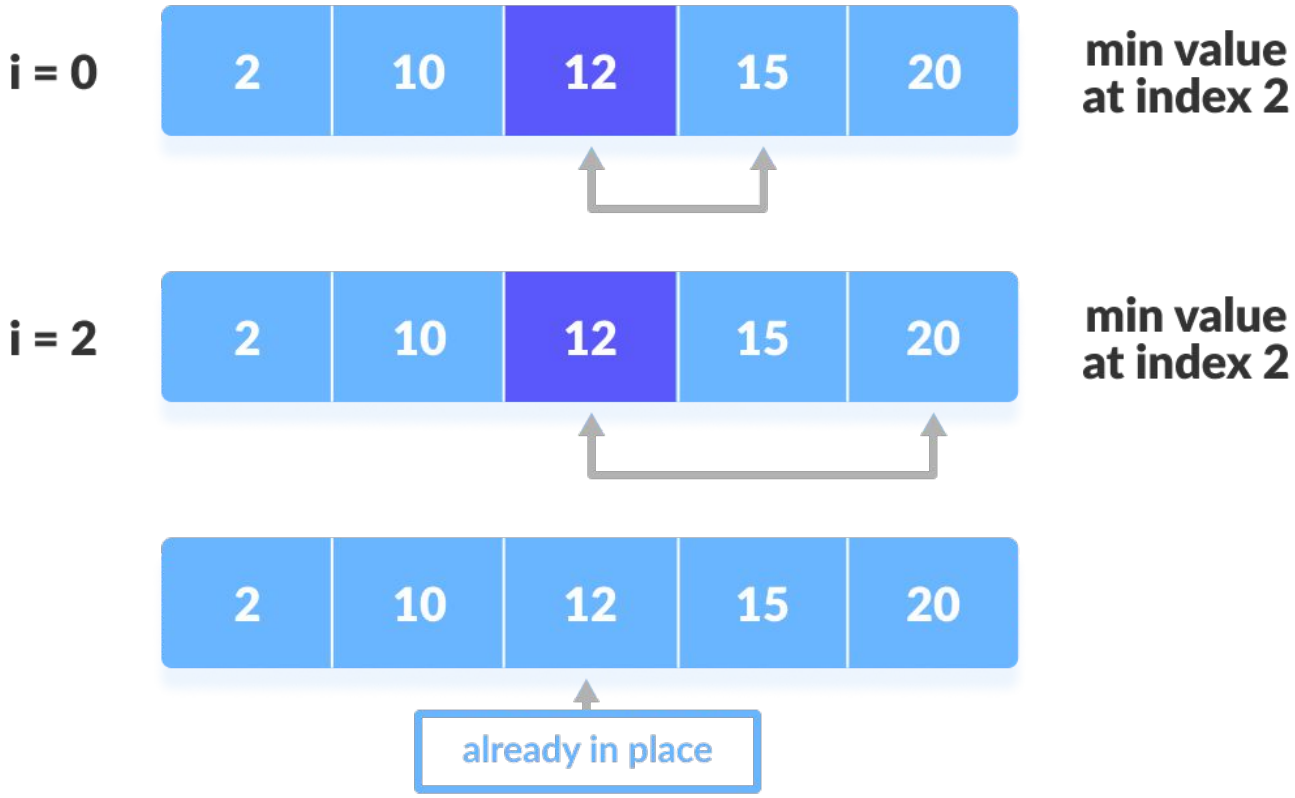
step = 0


i = 0
20 12 10 15 2
min value at index 1
i = 1
20 12 10 15 2
min value at index 2
i = 2
20 12 10 15 2
min value at index 2
i = 3
20 12 10 15 2
min value at index 4
2 12 10 15 20
swapping

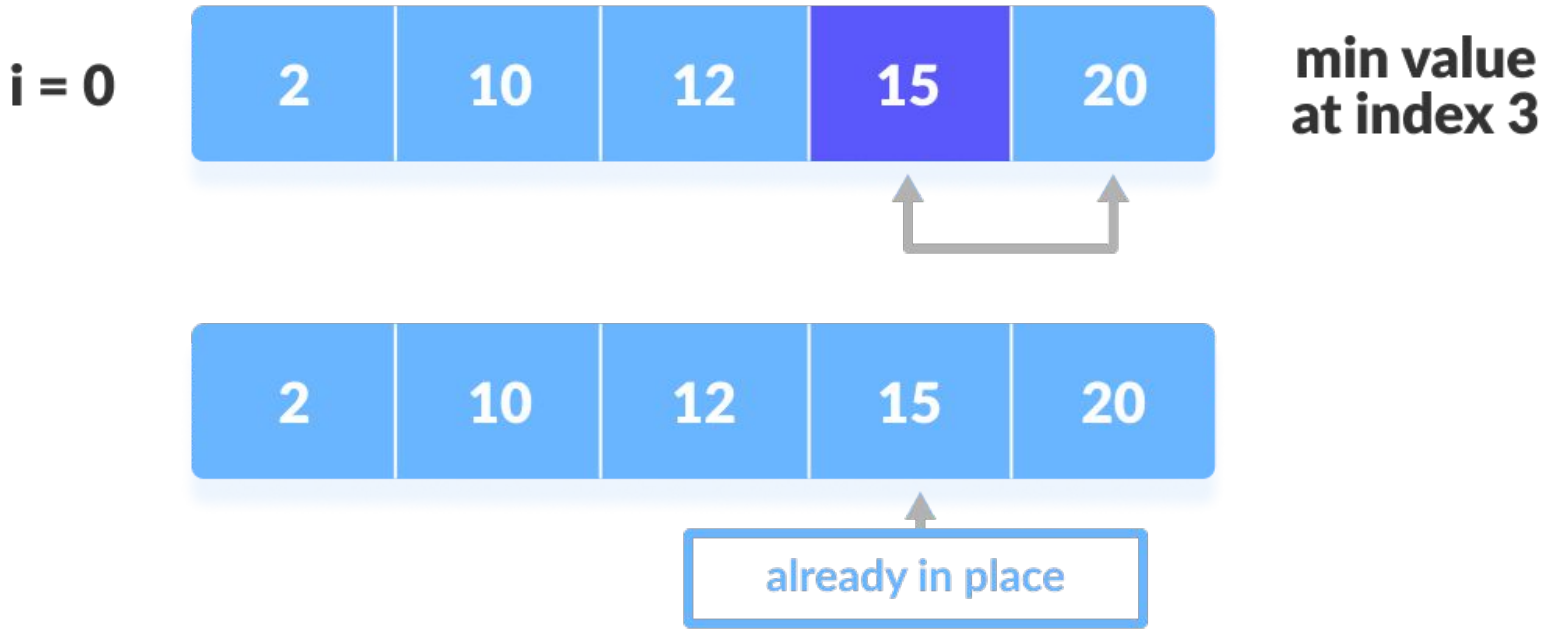
step = 1


i = 0
2 12 10 15 20
min value at index 2
i = 1
2 12 10 15 20
min value at index 2
i = 2
2 12 10 15 20
min value at index 2
2 10 12 15 20
swapping

step = 2


i = 0
2
10
12
15
20
min value
at index 2
i = 2
2
10
12
15
20
min value
at index 2
2
10
12
15
20
already in place

step = 3


i = 0
2
10
12
15
20
min value
at index 3
2
10
12
15
20
already in place

sele.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Start here | sele.c

```c
#include<stdio.h>
void main()
{
  int i,j,min,n,temp,a[20];
  printf("enter the Size of array:=\n");
  scanf("%d",&n);
   printf("enter array elements\n");
  for(i=0;i<n;i++)
  {
  scanf("%d",&a[i]);
  }
for(i=0;i<n-1;i++)
  {   min=i;
    for(j=i+1;j<n;j++)
    {
      if(a[j]<a[min])
       min=j;
      }
    temp=a[min];
    a[min]=a[i];
    a[i]=temp;
       }
    for(i=0;i<n;i++)
    {
    printf("%d\n",a[i]);
    }
     }
```

C:\Users\Samarth\Documents\sele.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 13, Col 13, Pos 247 | Insert | Read/Write | default

Type here to search | 29°C Sunny | ENG | 11:56 17-03-2026

BUBBLE SORT

Bubble sort is a sorting algorithm.

It works by repeatedly swapping adjacent elements if they are in the wrong order.

It sorts the array in ascending or descending order.

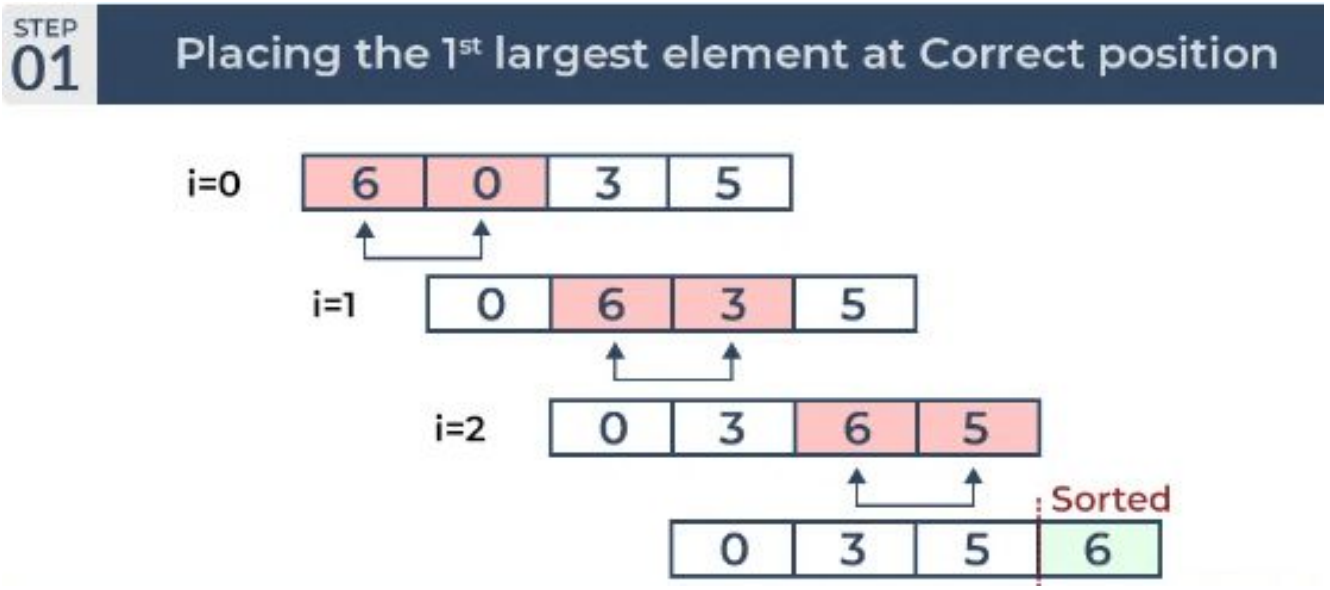
Working of Bubble Sort:

Let us understand the working of bubble sort with the help of the following illustration:

Input: *arr[] = {6, 3, 0, 5}*

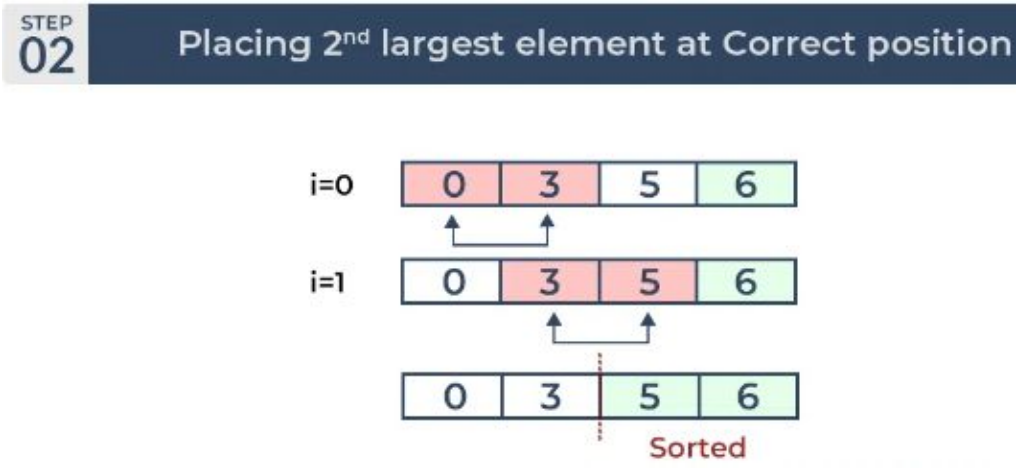
First Pass:

The largest element is placed in its correct position, i.e., the end of the array.



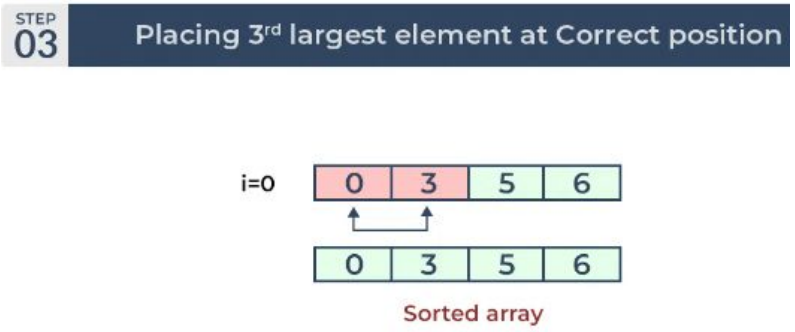
Second Pass:

Place the second largest element at correct position



Third Pass:

Place the remaining two elements at their correct positions



Total no. of passes: n-1

Total no. of comparisons: n*(n-1)/2

bb.c - Code::Blocks 25.03

```c
#include <stdio.h>
void swap(int *arr, int i, int j)
{
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {

        // Last i elements are already in place, so the loop will only num n - i - 1 times
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
                swap(arr, j, j + 1);
        }
    }
}
int main()
{
    int arr[] = {5, 6, 1, 3};
    int n = sizeof(arr) / sizeof(arr[0]);

    // Calling bubble sort on array arr
    bubbleSort(arr, n);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

C:\Users\Samarth\Documents\bb.c

STACK

A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle.

Stack has one end, whereas the Queue has two ends (**front and rear**). It contains only one pointer **top pointer** pointing to the topmost element of the stack. Whenever an element is added in the stack, it is added on the top of the stack, and the element can be deleted only from the stack. In other words, a ***stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.***

Some key points related to stack

It is called as stack because it behaves like a real-world stack, piles of books, etc.

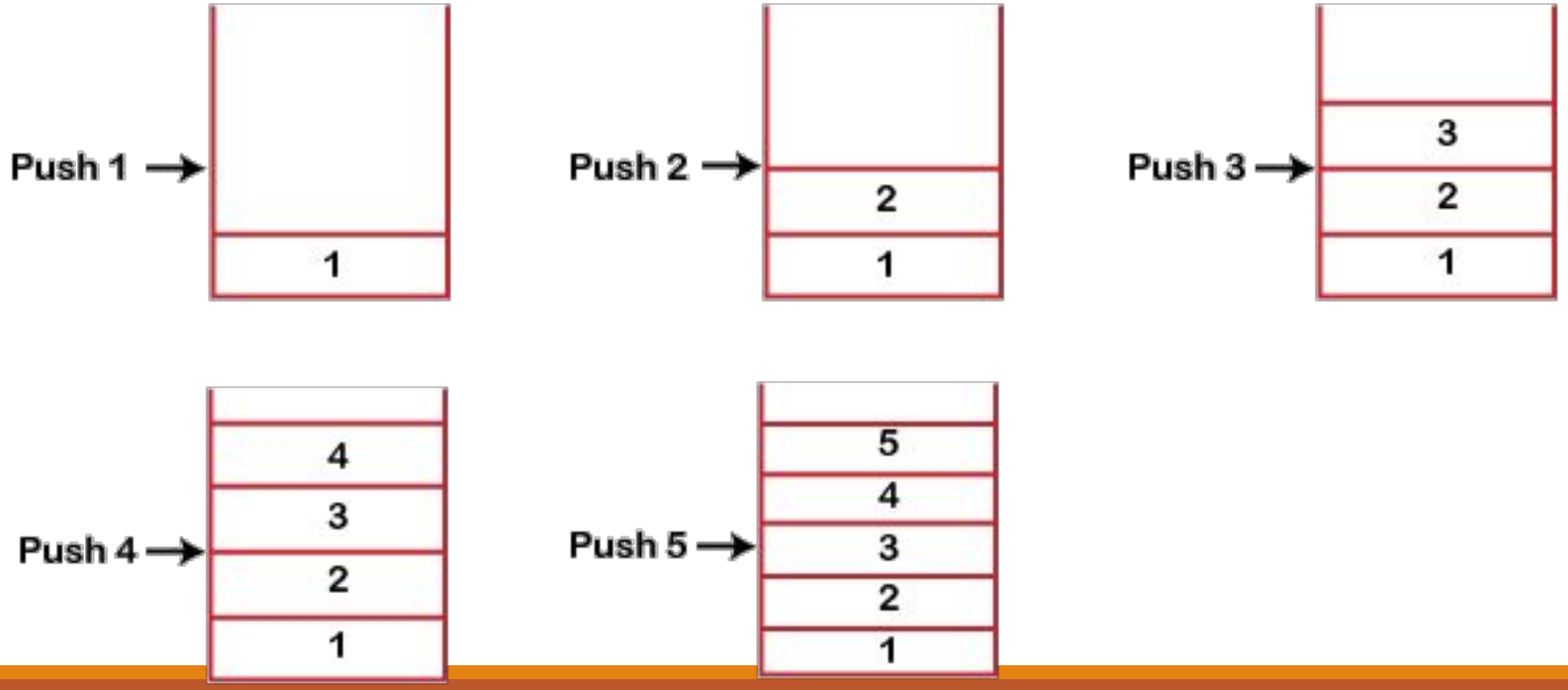
A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.

It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO or FILO.

Example

Stack works on the LIFO pattern. As we can observe in the below figure there are five memory blocks in the stack; therefore, the size of the stack is 5.

Suppose we want to store the elements in a stack and let's assume that stack is empty. We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.



The following are some common operations implemented on the stack:

push(): When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.

---

pop(): When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.

isEmpty(): It determines whether the stack is empty or not.

isFull(): It determines whether the stack is full or not.'

peek(): It returns the element at the given position.

count(): It returns the total number of elements available in a stack.

change(): It changes the element at the given position.

display(): It prints all the elements available in the stack.

PUSH operation

The steps involved in the PUSH operation is given below:

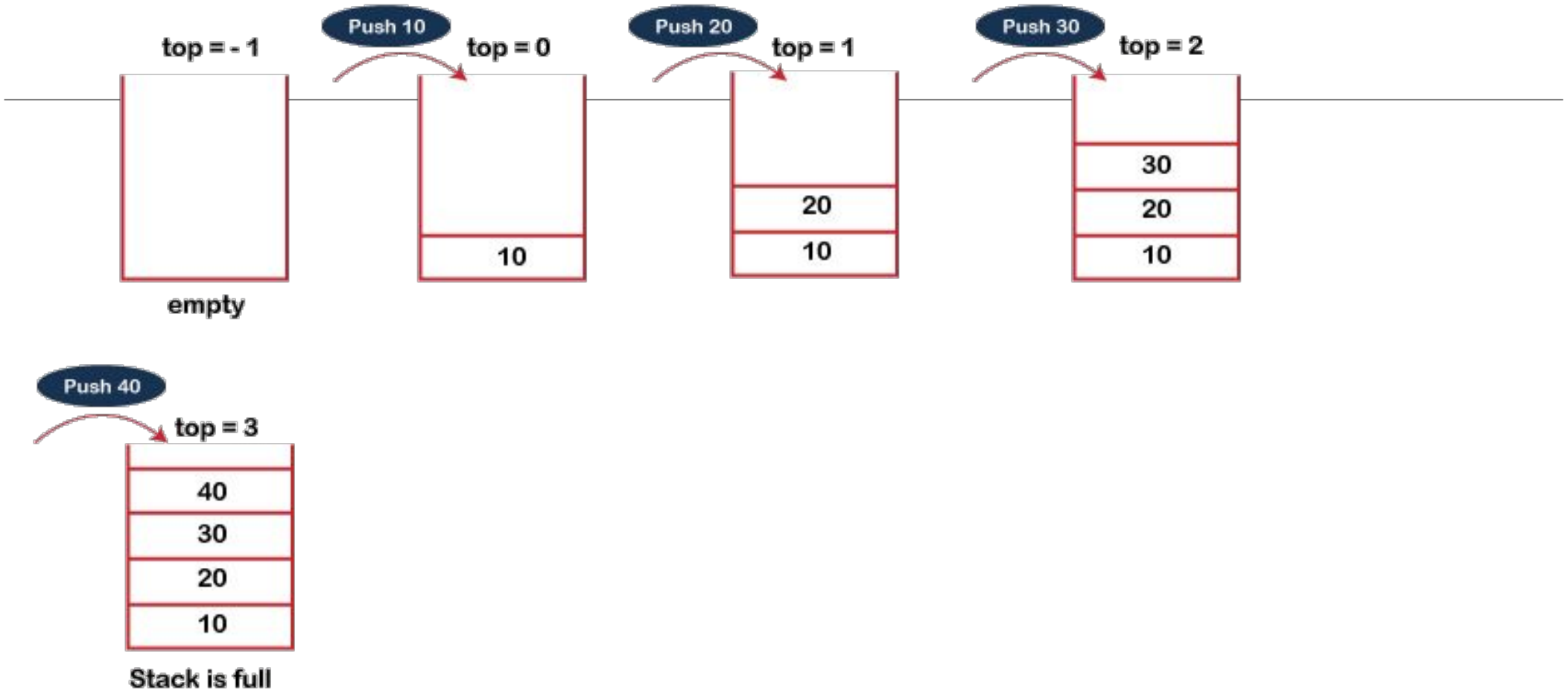
Before inserting an element in a stack, we check whether the stack is full.

If we try to insert the element in a stack, and the stack is full, then the ***overflow*** condition occurs.

When we initialize a stack, we set the value of top as -1 to check that the stack is empty.

When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1,** and the element will be placed at the new position of the **top**.

The elements will be inserted until we reach the ***max*** size of the stack


top = - 1
empty
Push 10
top = 0
10
Push 20
top = 1
20
10
Push 30
top = 2
30
20
10
Push 40
top = 3
40
30
20
10
Stack is full

stack.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> | main() : void

Start here | *sparse.c | stack.c

```c
#include<stdio.h>
int stack[5];
int top=-1;
void main()
{
void push();
void pop();
void peek();
void display();
int ch;
do
{
printf("Enter choice:\n1.push 2.pop 3.peek 4.display=");
scanf("%d",&ch);
switch(ch)
{
case 1: push();
    break;
case 2: pop();
    break;
case 3: peek();
    break;
case 4: display();
    break;
 default:
 printf("invalid choice");
 }
 }
 while(ch!=0);
 }
 void push()
 {
  int x;
  printf("Enter the data:");
  scanf("%d",&x);
  if(top==4)
 {
   printf("overflow");
 }
```

C:\Users\Samarth\Documents\stack.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 15, Col 11, Pos 223 | Insert | Read/Write | default

Type here to search | AIRTELPP -1.95% | ENG | 11:30 27-02-2026

stack.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Start here | *sparse.c | stack.c

```c
 top++;
 stack[top]=x;
}
}

 void pop()
{
 int item;
 if(top==-1)
{
 printf("underflow");
}
 else
{
 item=stack[top];
 top--;
 printf("poped item=%d\n",item);
}
}
void peek()
{
 if (top==-1)
 {
  printf("stack is empty");
 }
 else
  {
  printf("top most element is %d\t",stack[top]);
  }
  }
  void display()
  {
  int i;
  for(i=top;i>=0;i--)
  {
  printf("%d\t",stack[i]);
  }
  }
```

C:\Users\Samarth\Documents\stack.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 80, Col 1, Pos 993 | Insert | Read/Write | default

Type here to search | AIRTELPP -1.95% | ENG | 11:32 27-02-2026

"Recursion" is technique of solving any problem by calling same function again and again until some breaking (base) condition .

For eg. calculating factorial of a given number .

Thus in recursion last function called needs to be completed first.

Now Stack is a LIFO data structure i.e. (Last In First Out) and hence it is used to implement recursion.

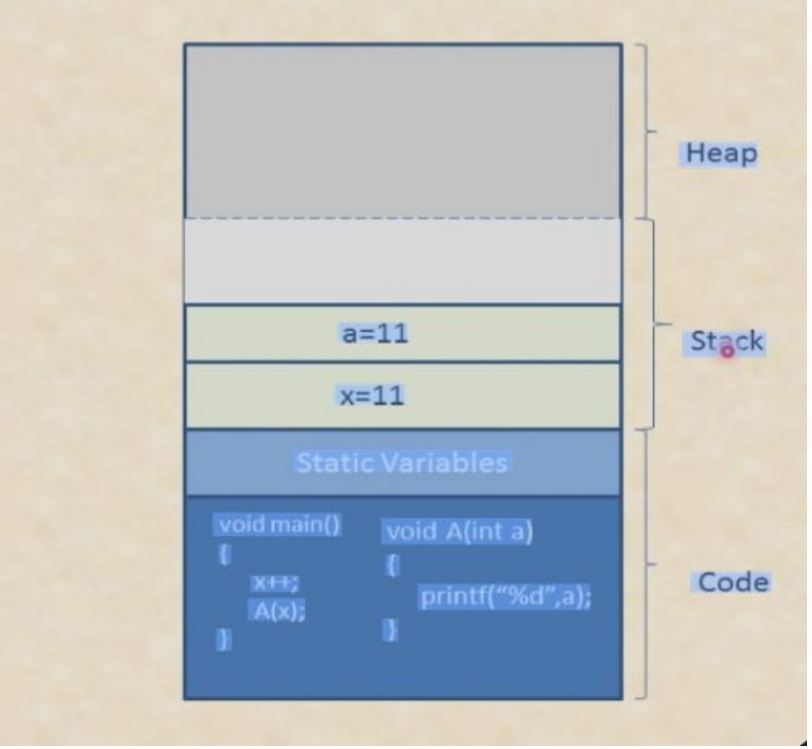
In each recursive call, there is need to save the

- current values of parameters,
- local variables and
- the return address (the address where the control has to return from the call).

Also, as a function calls to another function, first its arguments, then the return address and finally space for local variables is pushed onto the stack.

Recursion is extremely useful and extensively used because many problems are elegantly specified or solved in a recursive way.

The example of recursion as an application of stack is keeping books inside the drawer and the removing each book recursively.



Balancing Symbols

Balancing symbols in a data structure is a classic application of the **stack data structure** used to validate the syntax of expressions by ensuring all opening symbols have a matching closing symbol in the correct order .

The principle of symbol balancing (parentheses (), curly braces {}, and square brackets []) relies on the **Last-In, First-Out (LIFO)** property of the stack.

Algorithm :

Declare a structure for character stack.

Now traverse the expression string exp.

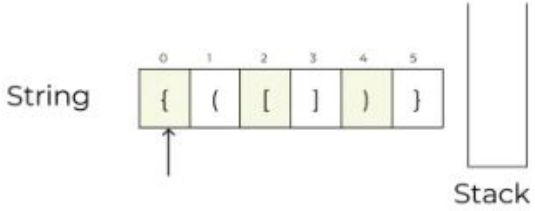
- If the current character is a starting bracket (**'(' or '{' or '['**) then push it to stack.
- If the current character is a closing bracket (**')' or '}' or ']'**) then pop from stack and if the popped character is the matching starting bracket then fine else brackets are not balanced.

After complete traversal, if there is some starting bracket left in stack then "NOT BALANCED"

Balanced Parenthesis Problem

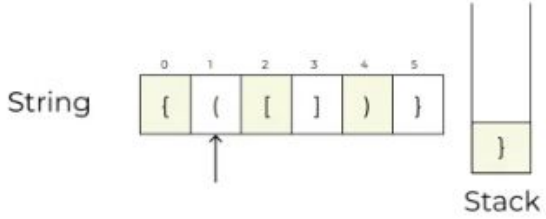
Initially

Opening brackets push into stack



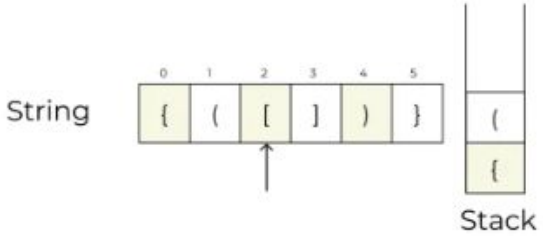
Step 1

Opening brackets push into stack



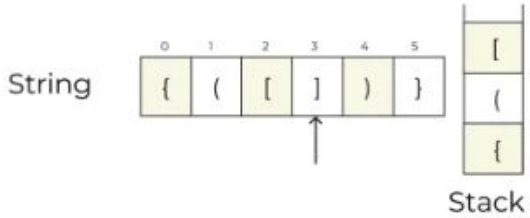
Step 2

Opening brackets push into stack



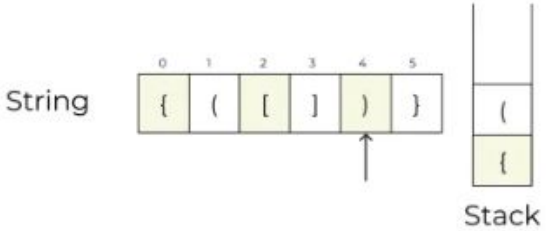
Step 3

Closing brackets Check top of the stack is same or not



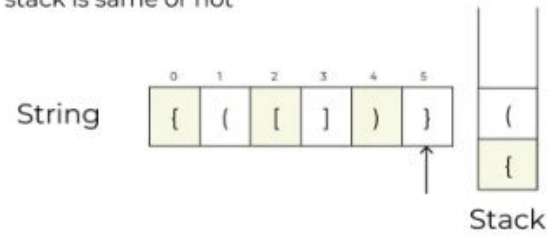
Step 4

Closing brackets Check top of the stack is same or not



Step 5

Closing brackets Check top of the stack is same or not



Comparison of Notations

Aspect	Infix Notation	Prefix Notation	Postfix Notation
Operator Placement	Between operands	Before operands	After operands
Parentheses Usage	Often required	Not required	Not required
Human Readability	Intuitive for humans	Less intuitive	Less intuitive
Computer Evaluation	Complex parsing needed	Simplified parsing	Simplified stack-based evaluation
Example	`(2 + 3) * 4`	`* + 2 3 4`	`2 3 + 4 *`

Operator	Precedence
Parentheses ()	Highest
Exponents ^	High
Multiplication *	Medium
Division /	Medium
Addition +	Low
Subtraction -	Low

A+B*C+D

a + b * c + d

Scanned character	Stack	Postfix Expression	Explaination
a		a	operand, put it in P.E
+	+	a	operator, push to stack
b	+	ab	operand, put it in P.E
*	+ *	ab	operator, push to stack
c	+ *	abc	operand, put it in P.E
+	+	abc*+	operator, push to stack
d	+	abc*+d	operand, put in P.E
		abc*+d+	

38

Queue

A queue is a linear data structure that follows the First In First Out (FIFO) order of insertion and deletion.

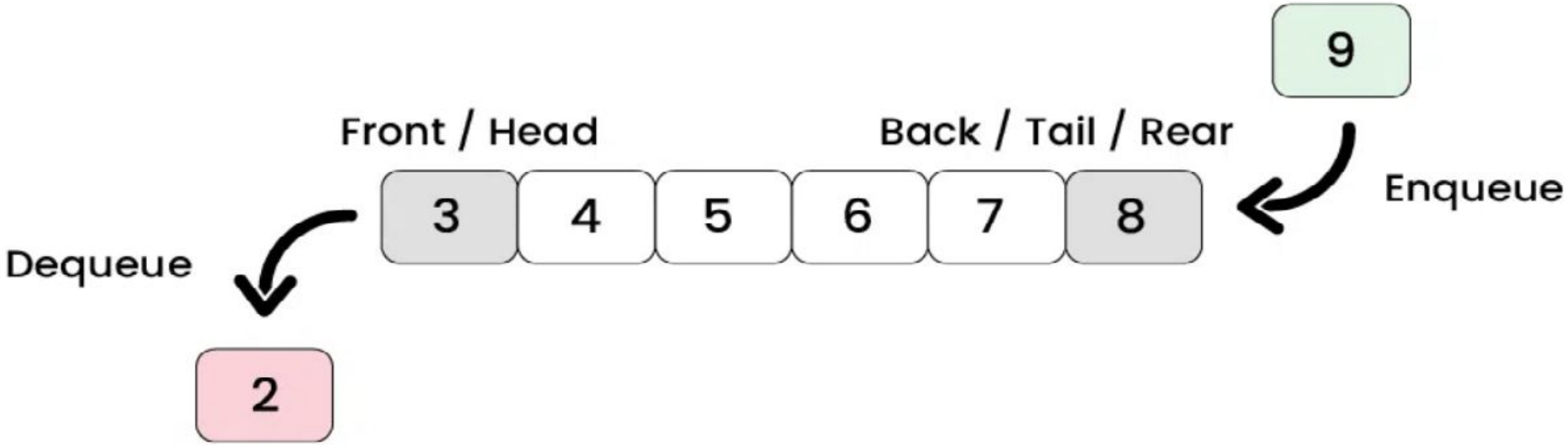
It means that the element that is inserted first will be the first one to be removed and the element that is inserted last will be removed at last.

Elements are inserted at the rear (enqueue) and removed from the front (dequeue).

Common operations are enqueue, dequeue, peek/front, isEmpty, and isFull.

Queues can be implemented using arrays or linked lists in C.

Queue in C



Basic Operations Queue operations may involve initializing or defining the queue, utilizing it, and then completely erasing it from the memory.

enqueue() − add (store) an item to the queue.

dequeue() − remove (access) an item from the queue. Few more functions are required to make the above-mentioned queue operation efficient.

peek() − Gets the element at the front of the queue without removing it. isfull() − Checks if the queue is full.

isempty() − Checks if the queue is empty.

queue.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> | main() : void

Start here | queue.c

```c
#include<stdio.h>
# define N 5
int queue[N];
int front=-1;
int rear=-1;
void main()
{
void enqueue();
void dequeue();
void display();
int ch;
do
{
printf("1.enqueue 2.dequeue 3.display=\nEnter choice:");
scanf("%d",&ch);
switch(ch)
{
case 1: enqueue();
    break;
case 2: dequeue();
    break;
case 3: display();
    break;
default:
 printf("invalid choice");
 }
 }
 while(ch!=0);
}
void enqueue()
{
 int x;
  printf("Enter the data:");
  scanf("%d",&x);
 if(rear==N-1)
  {
    printf("overflow");
  }
```

C:\Users\Samarth\Documents\queue.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 28, Col 15, Pos 398 | Insert | Read/Write | default

Type here to search | 35°C Mostly sunny | ENG | 13:51 23-03-2026

queue.c - Code::Blocks 25.03

```c
else if(front==-1 && rear==-1)
  {
   front=rear=0;
   queue[rear]=x;
   }
   else
   {
    rear++;
    queue[rear]=x;
    }
  }

 void dequeue()
 {
 if(front==-1&&rear==-1)
 {
 printf("underflow ");
 }
 else if(front==rear)
 {
  front=rear=-1;
  }
  else
  {
  printf("deque element is %d\n",queue[front]);
   front++;
   }
 }

void display()
{
 int i;
  if(front==-1&&rear==-1)
  {
  printf("queue empty");
  }
  else
  {
```

C:\Users\Samarth\Documents\queue.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 73, Col 5, Pos 1017 | Insert | Read/Write | default

Type here to search | 35°C Mostly sunny | ENG | 13:52 23-03-2026

queue.c - Code::Blocks 25.03

```c
    }

  void dequeue()
  {
  if(front==-1&&rear==-1)
  {
  printf("underflow ");
  }
  else if(front==rear)
  {
   front=rear=-1;
   }
   else
   {
   printf("deque element is %d\n",queue[front]);
    front++;
    }
  }

 void display()
 {
  int i;
   if(front==-1&&rear==-1)
   {
   printf("queue empty");
   }
   else
   {
   for(i=front;i<rear+1;i++)
   {
   printf("Enqueue data are %d\n",queue[i]);
   }
  }
 }
```

C:\Users\Samarth\Documents\queue.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 84, Col 1, Pos 1170 | Insert | Read/Write | default

Type here to search | 35°C Mostly sunny | ENG | 13:52 23-03-2026

Linked List

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations.

The elements in a linked list are linked using references.

In simple words, a linked list consists of nodes where each node contains a data field and a reference (link) to the next node in the list.

Both Arrays and Linked List can be used to store linear data of similar types, but they both have some advantages and disadvantages over each other, but arrays have following limitations.

1) The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.

2) Inserting a new element in an array of elements is expensive, because room has to be created for the new elements and to create room existing elements have to shifted.

For example, in a system if we maintain a sorted list of IDs in an array id[].

id[] = [1000, 1010, 1050, 2000, 2040].

And if we want to insert a new ID 1005, then to maintain the sorted order, we have to move all the elements after 1000 (excluding 1000).
Deletion is also expensive with arrays until unless some special techniques are used.

For example, to delete 1010 in id[], everything after 1010 has to be moved.

3) Due to the contiguous memory allocation, the size of an array must be predetermined. This usually leads to memory wastage .

There is no need to determine the size of a linked list. This not only avoids memory wastage but is very useful when a programmer is unaware of how large the linked list can grow.

There are four key types of linked lists:

- Singly linked lists
- Doubly linked lists
- Circular linked lists
- Circular doubly linked lists

Singly Linked List

The singly-linked list is a simple linked list, where each node points to the next node and the final node points to NULL.

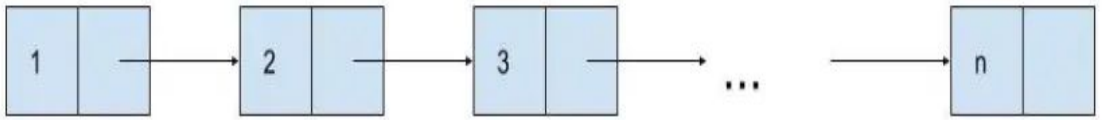


Figure 2: Single Linked List

```
struct node
{
   int data;
  struct node *next;
};
```

Doubly Linked List

A doubly linked list is the linked list in which each node points to both the next element and the previous element.

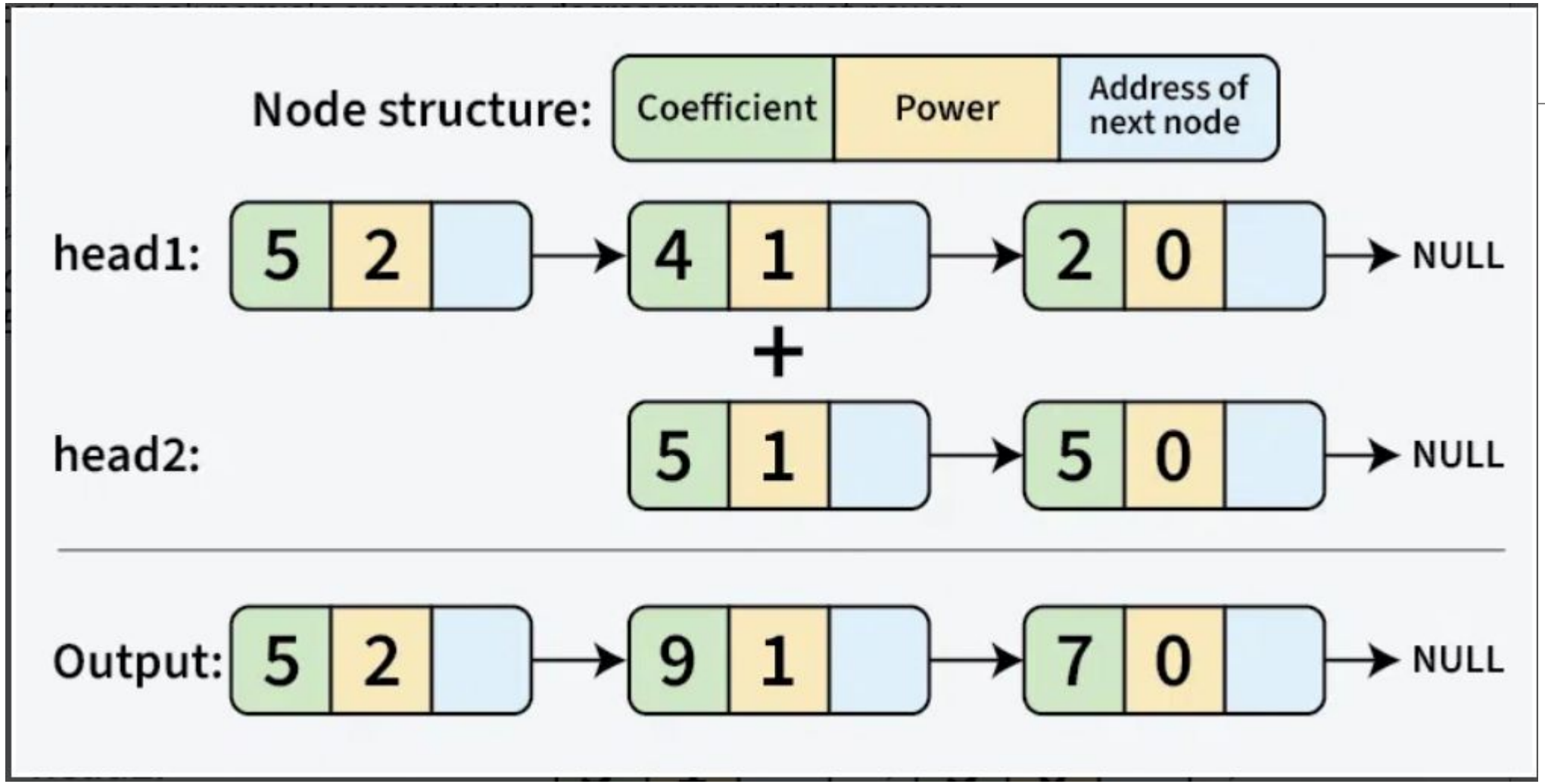
Note that the first element has its previous pointer to NULL and the last element has its next pointer to NULL.

Figure 3: Doubly Linked List

Adding two polynomials using Linked List

Two polynomial numbers represented by a linked list. The task is to **add** these lists meaning the coefficients with the same variable powers will be added.

Input:
head1: [[5, 2], [4, 1], [2, 0]]
head2: [[5, 1], [5, 0]]
Output: *[[5, 2], [9, 1], [7, 0]*


Node structure:
Coefficient
Power
Address of next node
head1:
5
2
4
1
2
0
NULL
+
head2:
5
1
5
0
NULL
Output:
5
2
9
1
7
0
NULL

Input:
head1: [[5, 3], [4, 2], [2, 0]]
head2: [[5, 1], [-5, 0]]
Output: *[[5, 3], [4, 2], [5, 1], [-3, 0]]*
Explanation: *head1 can be represented as* ***5x^3 + 4x^2 + 2*** *, head2 can be represented as* ***5x - 5****, add both the polynomials to get* ***5x^3 + 4x^2 + 5x - 3***

Input: *A[] = [5, 0, 10, 6]*
B[] = [1, 2, 4]
Output: *[5, 10, 30, 26, 52, 24]*

The first input array represents "$5 + 0x^1 + 10x^2 + 6x^3$"
The second array represents "$1 + 2x^1 + 4x^2$"
after Multiply both them we get "$5 + 10x^1 + 30x^2 + 26x^3 + 52x^4 + 24x^5$"

Multiplication of two polynomials using Linked list

Given **two polynomials** in the form of a **linked list,** the task is to perform the **multiplication** of both polynomials.

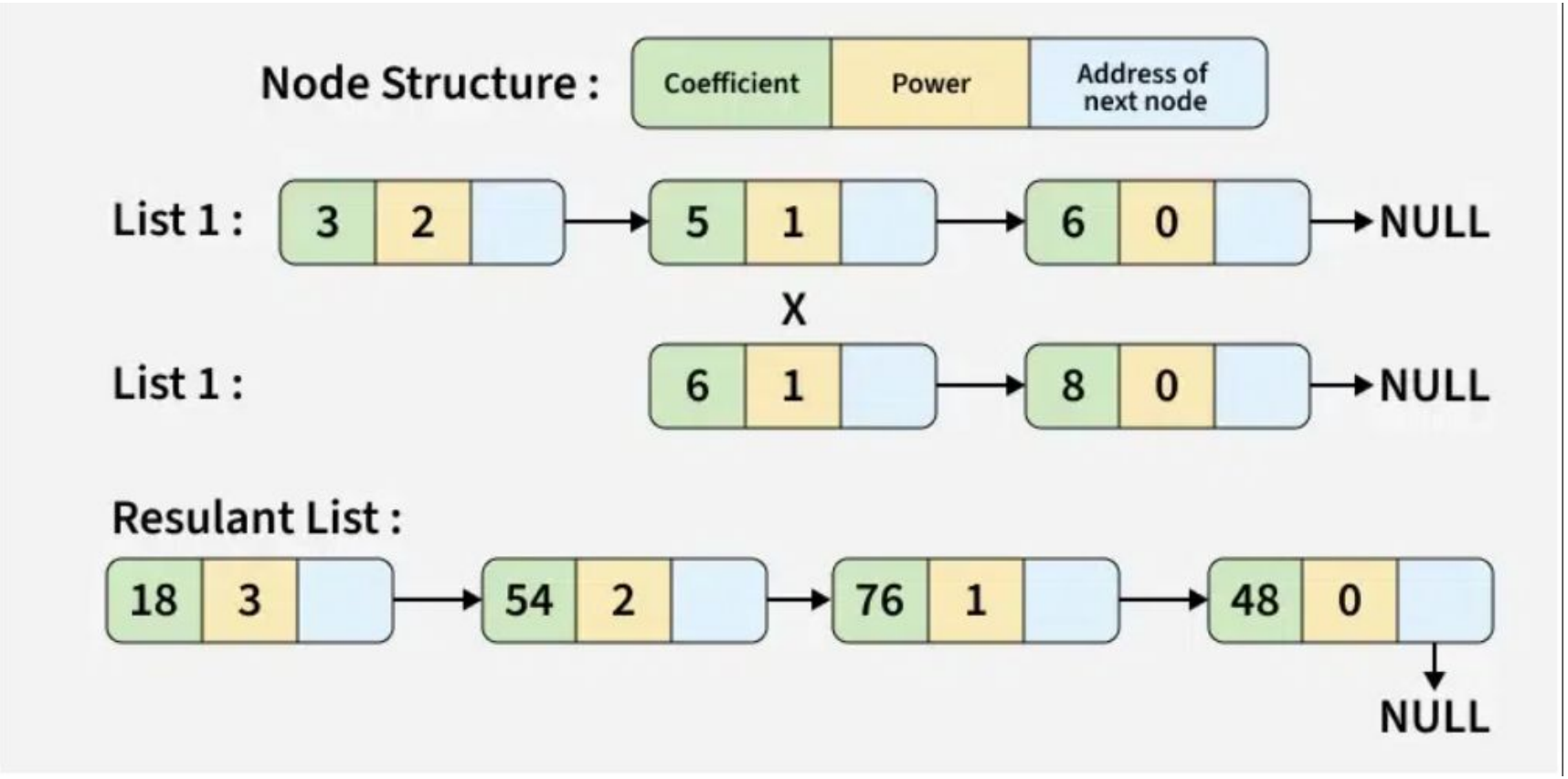
Examples:

Input:

poly1: 3x^2 + 5x^1 + 6,

poly2: 6x^1 + 8
Output: *18x^3 + 54x^2 + 76x^1 + 48*

Explanation: *On multiplying each element of 1st polynomial with elements of 2nd polynomial, we get*
18x^3 + 24x^2 + 30x^2 + 40x^1 + 36x^1 + 48
On adding values with same power of x,
18x^3 + 54x^2 + 76x^1 + 48


Node Structure :
Coefficient
Power
Address of next node
List 1 :
3
2
5
1
6
0
NULL
X
List 1 :
6
1
8
0
NULL
Resulant List :
18
3
54
2
76
1
48
0
NULL

Input: *poly1: 3x^3 + 6x^1 - 9, poly2: 9x^3 - 8x^2 + 7x^1 + 2*
Output: *27x^6 - 24x^5 + 75x^4 - 123x^3 + 114x^2 - 51x^1 - 18*

Thank You !